

CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY

FACULTY OF TECHNOLOGY & ENGINEERING

SMT. KUNDANBEN DINSHA PATEL DEPARTMENT OF INFORMATION
TECHNOLOGY

Subject Name: Python Programming
Subject Code: IT374

Semester: B.Tech V
Academic year: June -Dec 2019

Python Lab Manual 2019-20

Important tips:

Trick #1

Reversing a string in Python

```
>>> a = "codementor"
>>> print "Reverse is",a[::-1]
Reverse is rotnemedoc
```

Trick #2

Transposing a Matrix

```
>>> mat = [[1, 2, 3], [4, 5, 6]]
>>> zip(*mat)
[(1, 4), (2, 5), (3, 6)]
```

Trick #3

a = [1,2,3]

Store all three values of the list in 3 new variables

```
>>> a = [1, 2, 3]
>>> x, y, z = a
>>> x
1
>>> y
2
>>> z
3
```

Trick #4

```
a = ["Code", "mentor", "Python", "Developer"]
```

Create a single string from all the elements in list above.

```
>>> print " ".join(a)
Code mentor Python Developer
```

Trick #5

List 1 = ['a', 'b', 'c', 'd']

List 2 = ['p', 'q', 'r', 's']

Write a Python code to print

- ap
- bq
- cr
- ds

```
>>> for x, y in zip(list1, list2):
...     print x, y
...
a p
b q
c r
d s
```

Trick #6

Swap two numbers with one line of code.

```
>>> a=7
>>> b=5
>>> b, a =a, b
>>> a
5
>>> b
7
```

Trick #7

Print "codecodecodecode mentormentormentormentormentor" without using loops

```
>>> print "code"*4+' '+ "mentor"*5
```

codecodecodecode mentormentormentormentormentor

Trick #8

`a = [[1, 2], [3, 4], [5, 6]]`

Convert it to a single list without using any loops.

Output:- `[1, 2, 3, 4, 5, 6]`

```
>>> import itertools
>>> list(itertools.chain.from_iterable(a))
[1, 2, 3, 4, 5, 6]
```

Trick #9

Checking if two words are anagrams

```
def is_anagram(word1, word2):
    """Checks whether the words are anagrams.
    word1: string
    word2: string
    returns: boolean
    """
```

Complete the above method to find if two words are anagrams.

```
from collections import Counter
def is_anagram(str1, str2):
    return Counter(str1) == Counter(str2)
>>> is_anagram('abcd', 'dbca')
True
>>> is_anagram('abcd', 'dbaa')
False
```

Trick #10.

Taking a string input.

For example "1 2 3 4" and return `[1, 2, 3, 4]`

Remember list being returned has integers in it.

Don't use more than one line of code.

```
>>> result = map(lambda x:int(x) ,raw_input().split())
1 2 3 4
>>> result
```

```
, 2, 3, 4]
```

Practical No 1:	Create a program that asks the user to enter their name and their age. Print out a message addressed to them that tells them the year that they will turn 100 years old.
Solution :	<pre>name = input("What is your name: ") age = int(input("How old are you: ")) year = str((2014 - age)+100) print(name + " will be 100 years old in the year " + year)</pre>
Practical No 2:	Ask the user for a number. Depending on whether the number is even or odd, print out an appropriate message to the user. Hint: how does an even / odd number react differently when divided by 2? Self: If the number is a multiple of 4, print out a different message.
Solution :	<pre>num = input("Enter a number: ") mod = num % 2 if mod > 0: print("You picked an odd number.") else: print("You picked an even number.")</pre>

List

A list is a data structure that holds an ordered collection of items i.e. you can store a sequence of items in a list. This is easy to imagine if you can think of a shopping list where you have a list of items to buy, except that you probably have each item on a separate line in your shopping list whereas in Python you put commas in between them.

```
# This is my shopping list
shoplist = ['apple', 'mango', 'carrot', 'banana']

print('I have', len(shoplist), 'items to purchase.')

print('These items are:', end=' ')
for item in shoplist:
    print(item, end=' ')

print('\nI also have to buy rice.')
shoplist.append('rice')
print('My shopping list is now', shoplist)

python ds_using_list.py
```

```
I have 4 items to purchase.
These items are: apple mango carrot banana
I also have to buy rice.
My shopping list is now ['apple', 'mango', 'carrot', 'banana', 'rice']
```

Tuple

Tuples are used to hold together multiple objects. Think of them as similar to lists, but without the extensive functionality that the list class gives you. One major feature of tuples is that they are immutable like strings i.e. you cannot modify tuples.

```
zoo = ('python', 'elephant', 'penguin')
print('Number of animals in the zoo is', len(zoo))

new_zoo = 'monkey', 'camel', zoo    # parentheses not required but are a good
idea
print('Number of cages in the new zoo is', len(new_zoo))
print('All animals in new zoo are', new_zoo)
print('Animals brought from old zoo are', new_zoo[2])
print('Last animal brought from old zoo is', new_zoo[2][2])
print('Number of animals in the new zoo is',
      len(new_zoo)-1+len(new_zoo[2]))
```

Output:

```
$ python ds_using_tuple.py
Number of animals in the zoo is 3
Number of cages in the new zoo is 3
All animals in new zoo are ('monkey', 'camel', ('python', 'elephant', 'penguin'))
Animals brought from old zoo are ('python', 'elephant', 'penguin')
Last animal brought from old zoo is penguin
Number of animals in the new zoo is 5
```

Dictionary

A dictionary is like an address-book where you can find the address or contact details of a person by knowing only his/her name i.e. we associate keys (name) with values (details). Note that the key must be unique just like you cannot find out the correct information if you have two persons with the exact same name.

```
ab = {
    'Swaroop': 'swaroop@swaroopch.com',
    'Larry': 'larry@wall.org',
    'Matsumoto': 'matz@ruby-lang.org',
    'Spammer': 'spammer@hotmail.com'
}

print("Swaroop's address is", ab['Swaroop'])

# Deleting a key-value pair
del ab['Spammer']

print('\nThere are {} contacts in the address-book\n'.format(len(ab)))

for name, address in ab.items():
    print('Contact {} at {}'.format(name, address))

# Adding a key-value pair
ab['Guido'] = 'guido@python.org'
```

```
if 'Guido' in ab:
    print("\nGuido's address is", ab['Guido'])
```

Output:

```
$ python ds_using_dict.py
Swaroop's address is swaroop@swaroopch.com
```

There are 3 contacts in the address-book

```
Contact Swaroop at swaroop@swaroopch.com
Contact Matsumoto at matz@ruby-lang.org
Contact Larry at larry@wall.org
```

```
Guido's address is guido@python.org
```

Set

Sets are *unordered* collections of simple objects. These are used when the existence of an object in a collection is more important than the order or how many times it occurs.

Using sets, you can test for membership, whether it is a subset of another set, find the intersection between two sets, and so on.

```
>>> bri = set(['brazil', 'russia', 'india'])
>>> 'india' in bri
True
>>> 'usa' in bri
False
>>> bric = bri.copy()
>>> bric.add('china')
>>> bric.issuperset(bri)
True
>>> bri.remove('russia')
>>> bri & bric # OR bri.intersection(bric)
{'brazil', 'india'}
```

Practical
7:

Write a password generator in Python. Be creative with how you generate passwords - strong passwords have a mix of lowercase letters, uppercase letters, numbers, and symbols. The passwords should be random, generating a new password every time the user asks for a new password. Include your run-time code in a main method.

Self:

Ask the user how strong they want their password to be. For weak passwords, pick a word or two from a list.

Solution

```
# generate a password with length "passlen" with no duplicate characters in the password
```

```
import random
```

```
s =
```

	<pre>"abcdefghijklmnopqrstuvwxyz01234567890ABCDEFGHIJKLMN OPQRSTUVWXYZ!@#\$%^&*()?" passlen = 8 p = "".join(random.sample(s,passlen)) print p</pre>
Practical 8:	Implement Multiple Inheritance. Python supports classes inheriting from other classes. The class being inherited is called the Parent or Superclass, while the class that inherits is called the Child or Subclass. How can we define the order in which the base classes are searched when executing a method? (Hint: MRO: Method Resolution Order)
Solution	<pre># first parent class class Person(object): def __init__(self, name, idnumber): self.name = name self.idnumber = idnumber # second parent class class Employee(object): def __init__(self, salary, post): self.salary = salary self.post = post # inheritance from both the parent classes class Leader(Person, Employee): def __init__(self, name, idnumber, salary, post, points): self.points = points Person.__init__(self, name, idnumber) Employee.__init__(self, salary, post) print(self.salary) ins = Leader('Rahul', 882016, 'Assistant Manager', 75000, 560)</pre> Output: <pre>Assistant Manager</pre>
Practical 8.2	Write a function that takes an ordered list of numbers (a list where the elements are in order from smallest to largest) and another number. The function decides whether or not the given number is inside the list and returns (then prints) an appropriate boolean.
Solution	<pre>def find(ordered_list, element_to_find): for element in ordered_list: if element == element_to_find: return True return False if __name__=="__main__":</pre>

	<pre> l = [2, 4, 6, 8, 10] print(find(l, 5)) # prints False print(find(l, 10)) # prints True print(find(l, -1)) # prints False print(find(l, 2)) # prints True </pre>
Practical 9:	Given a .txt file that has a list of a bunch of names, count how many of each name there are in the file, and print out the results to the screen.
Soulution	<pre> counter_dict = {} with open('nameslist.txt') as f: line = f.readline() while line: line = line.strip() if line in counter_dict: counter_dict[line] += 1 else: counter_dict[line] = 1 line = f.readline() print(counter_dict) </pre>
Practical 10:	Introduction to Django- Python based free and open-source web framework and Flask- Python based micro web framework.
Solution	Why Django A web framework is a collection of modular tools that abstracts away much of the difficulty—and repetition—inherent in web development. For example, most websites need the same basic functionality: the ability to connect to a database, set URL routes, display content on a page, handle security properly, and so on. Rather than recreate all of this from scratch, programmers over the years have created web frameworks in all the major programming languages: Django and Flask in Python, Rails in Ruby, and Express in JavaScript among many, many others. Django inherited Python’s “batteries-included” approach and includes out-of-the box support for common tasks in web development: - user authentication - templates, routes, and views - admin interface - robust security - support for multiple database backends

and much much more

This approach makes our job as web developers much, much easier. We can focus on what makes our web application unique rather than reinventing the wheel when it comes to standard web application functionality.

In contrast, several popular frameworks—most notably Flask in Python and Express in JavaScript—adopt a “microframework” approach. They provide only the bare minimum required for a simple web page and leave it up to the developer to install and configure third-party packages to replicate basic website functionality. This approach provides more flexibility to the developer but also yields more opportunities for mistakes.

As of 2019 Django has been under active development for over 14 years which makes it a grizzled veteran in software years. Millions of programmers have already used Django to build their websites. And this is undeniably a good thing. Web development is hard. It doesn't make sense to repeat the same code—and mistakes—when a large community of brilliant developers has already solved these problems for us.

At the same time, Django remains under active development and has a yearly release schedule. The Django community is constantly adding new features and security improvements. If you're building a website from scratch Django is a fantastic choice.

Link: <https://wiki.python.org/moin/WebFrameworks>